SY B.Tech. - ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

PRACTICAL ASSIGNMENT 12

1. Student Details

Student Name	Swanit Bhawar
PRN	202501110203
Division	C
Batch	C2
Roll No.	202501110203
Date of Submission	02-11-2026

2. Practical Details

Practical No.	12
Practical Title	TYPE CASTING AND CASTING OPERATORS
Date Performed	29-10-2026
Faculty Name	Ms Shruti

3. Problem Statement

Implement and demonstrate C++ casting operators static_cast, dynamic_cast, const_cast, and reinterpret_cast with suitable examples of numeric conversion, safe downcasting, const qualification and pointer conversion.

4. Objectives

1. Understand the purpose of explicit type casting in C++.
2. Implement all four C++ casting operators.
3. Verify the resulting behavior through program output.

5. Expected Learning Outcomes

1. Perform an explicit numeric conversion using static_cast.
2. Perform a runtime-checked downcast using dynamic_cast.
3. Explain const_cast and reinterpret_cast with appropriate usage.

6. Theory / Concept

Type casting converts a value or expression from one type to another. **static_cast** performs explicitly permitted conversions at compile time. **dynamic_cast** checks downcasts in a polymorphic class hierarchy at runtime. **const_cast** adds or removes const qualification; changing an originally const object is not permitted. **reinterpret_cast** performs low-level pointer and integer conversions subject to the implementation.

7. Algorithm / Pseudocode

1. Convert a floating-point number to an integer using static_cast.
2. Define base and derived classes and safely downcast a base pointer using dynamic_cast.
3. Remove const qualification from a pointer to a non-const integer and update its value.
4. Convert an integer pointer to uintptr_t and restore the pointer using reinterpret_cast.
5. Display the results of all four operations.

8. Technology / Tools / Libraries Used

Programming Language	C++
IDE / Platform	Visual Studio Code / Windows
Compiler	g++
Libraries	iostream, cstdint

9. Implementation / GitHub Repository

Repository Name	Swanit95/Object-Oriented-Programming-
Repository URL	https://github.com/Swanit95/Object-Oriented-Programming-
Folder	assignments/Assignment 12
Branch	main
Commit ID	ff279743ef3c9ea1518b934230a8ca9cc3c115bb

Implementation

Type Casting and Casting Operators



```cpp
#include <iostream>
using namespace std;

// Base class for dynamic_cast demonstration
class Animal {
public:
    virtual void sound() {
        cout << "Animal makes a sound" << endl;
    }

    virtual ~Animal() = default;
};

// Derived class
class Dog : public Animal {
public:
    void sound() override {
        cout << "Dog barks" << endl;
    }

    void display() {
        cout << "Downcasting successful" << endl;
    }
};

int main() {
    // Example 1: static_cast
    // Explicit conversion from double to int
    double num = 25.75;
    int result = static_cast<int>(num);

    cout << "Example 1: static_cast" << endl;
    cout << "Original value: " << num << endl;
    cout << "Converted value: " << result << endl;

    // Example 2: dynamic_cast
    // Safe downcasting from base class to derived class
    cout << "\nExample 2: dynamic_cast" << endl;

    Dog d;
    Animal* basePtr = &d;

    Dog* derivedPtr = dynamic_cast<Dog*>(basePtr);

    if (derivedPtr != nullptr) {
        derivedPtr->sound();
        derivedPtr->display();
    } else {
        cout << "Downcasting failed" << endl;
    }

    // Example 3: const_cast
    // Remove const qualification from a pointer
    cout << "\nExample 3: const_cast" << endl;

    int value = 50;
    const int* ptr = &value;

    int* newPtr = const_cast<int*>(ptr);
    *newPtr = 100;

    cout << "Modified value: " << value << endl;

    // Example 4: reinterpret_cast
    // Convert a pointer to an integer representation
    cout << "\nExample 4: reinterpret_cast" << endl;

    int number = 10;
    int* p = &number;

    uintptr_t address = reinterpret_cast<uintptr_t>(p);
    int* restoredPtr = reinterpret_cast<int*>(address);

    cout << "Original value: " << number << endl;
    cout << "Value through restored pointer: "
         << *restoredPtr << endl;

    return 0;
}
```

10. Test Cases / Input and Output

Example	Expected Output	Actual Output	Status
static_cast	25	25	Pass
dynamic_cast	Dog barks; Downcasting successful	Matched	Pass
const_cast	Modified value: 100	Matched	Pass
reinterpret_cast	Restored value: 10	Matched	Pass

11. Results / Observations

The program compiled and executed successfully. All four casting demonstrations produced the expected values.



12. Conclusion

The practical demonstrated static_cast, dynamic_cast, const_cast, and reinterpret_cast and verified their use in explicit conversions and pointer operations.

13. Student Declaration

I hereby declare that the above practical work has been completed by me and that the implementation, analysis, and answers are my own work.

Student Name	PRN	Signature
Swanit Bhawar	202501110203	